GigBid

A human-controlled multi-agent freelancer copilot

AI-4016 R&D; Assignment - Engineering Design Review
LangGraph, advanced RAG, MCP, FastAPI, Next.js, Postgres, Redis and open models

Author	Course	Prepared	Report scope
Taaha Khan	AI-4016 Agentic AI	12 November 2026	Local evidence plus a live Render reviewer deployment

Executive summary

GigBid discovers jobs, evaluates fit and risk, prices work, retrieves verifiable portfolio evidence, drafts cited proposals, and pauses before every marketplace or message write. The implementation owns three MCP servers, places GigSim and Upwork behind one port, persists LangGraph interrupts in Postgres, streams run checkpoints to a responsive UI, and moves long runs and saved searches to Celery.

The strongest retrieval finding was negative: requirement decomposition reduced Recall@5, and lexical-only ranking beat type-diverse hybrid RRF on the authored synthetic benchmark. V2 still ships for maximum Recall@5, semantic/type coverage and claim grounding. A separate measured writer study selected Qwen3 4B over Llama 3.1 8B. A Render reviewer deployment is live; no human, hidden-test or live-marketplace result is invented.

Live reviewer: https://gigbid-web.onrender.com. Evidence: evals/results/*.json; docs/RAG_EXPERIMENT.md; docs/DEPLOYMENT_EVIDENCE.md

1. Problem, constraints and architecture

Freelancers waste time on weak jobs, spend scarce Connects, and risk sending generic or non-compliant proposals. GigBid may automate discovery, analysis and drafting; it must never auto-send a proposal or message, fabricate experience, scrape a platform, follow instructions embedded in marketplace content, or spend Connects without a human interrupt.

Next.js UI	**FastAPI**	**Celery + Redis**	**LangGraph**
Approval + SSE	**Typed API + auth**	**Durable long runs**	**Bid + conversation**
Portfolio MCP	**Marketplace MCP**	**Workspace MCP**	**Postgres + RAG**

State and trust boundaries

Job state contains only the current typed hand-offs. Execution history is checkpointed and traced. Long-term style and outcome memories are stored only after explicit promotion thresholds. Marketplace effects cross MarketplacePort; the graph cannot tell whether the adapter is memory, the self-contained simulator, or official Upwork MCP.

Invariant	Implementation	Verification
Human before writes	interrupt() plus dry-run approval payload	Approval, edit, reject and exactly-once tests
Grounded claims	Evidence IDs plus post-draft verification	Seeded unsupported claim: 1 before, 0 after
Durability	Postgres saver/store and idempotency keys	Restart/resume integration test
Budgets	Step, token, price and Connects limits	Needs-attention and profile-boundary tests
Isolation	Tenant/user namespaces and bearer auth	Cross-scope checks and private portfolio store

Declared targets: p95 job analysis <= 45 s; p95 proposal <= 90 s; open-profile model cost USD 0 locally. The deterministic simulator study measured sub-0.01 s graph latency, but this is not an LLM production latency claim. [agent/config.yaml; evals/results/simulator_outcomes.json]

2. R&D; A - MCP tool design

Hypothesis: fine-grained tools with specific descriptions improve correct selection and arguments over server-sized coarse tools, at a token cost. Forty known-call tasks were run through three designs using the same local llama3.1:8b model, temperature 0 and seed 4016. The study covers 120 calls and reports exact tool/argument matches.

Design	Tool accuracy	Argument accuracy	Tokens/task	p95 ms
Fine-grained	1.00	0.60	1229.6	5485
Coarse	0.80	0.15	411.0	6061
Best descriptions	1.00	0.50	1424.0	7740

Decision

Ship single-purpose tools with strict Pydantic inputs. Fine-grained tools tied for perfect selection and produced the best argument accuracy. Rewritten descriptions did not improve arguments and increased tokens, so description length alone is not a fix. The result is one-model evidence, not a universal model conclusion.

Owned and third-party surface

Server	Surface	Controls
Portfolio	search evidence, case study, skills; resources and prompt	Read-only, bearer auth, tenant metadata
Marketplace	jobs, clients, Connects, messages, proposal/message writes	Dry run, idempotency, rate limit, audit log
Workspace	slots, tentative holds, email drafts	Local mock, approval before writes
Upwork + GitHub	Official remote MCP endpoints	Encrypted token boundary, allowlisted tools, no scraping

Sources: evals/results/tool_design.json; docs/TOOL_DESIGN_EXPERIMENT.md; docs/TOOLS.md; tests/contract/

3. R&D; B - Evidence retrieval and verification

Ninety authored synthetic jobs (30 per persona) were evaluated against synthetic portfolio evidence. Each job has ten relevant items, so Recall@5 has a ceiling of 0.5. The preregistered hypothesis that V1, V2 and V3 all match or beat V0 failed.

Variant	Recall@5	MRR	nDCG@10	Attack unsupported before/after	p95 ms
V0 dense	0.494	0.989	0.738	1 / 0	2.5
V1 requirements	0.378	0.664	0.662	1 / 0	17.0
V2 hybrid	0.500	1.000	0.801	1 / 0	120.4
V3 rerank	0.498	1.000	0.749	1 / 0	120.4
V8 corrective	0.378	0.664	0.662	1 / 0	17.1

Ablation and shipped choice

Configuration	Recall@5	nDCG@10	p95 ms
Dense only	0.493	0.742	2.5
Lexical only	0.500	0.833	180.2
Hybrid + type cap	0.500	0.801	182.6

Lexical-only beat type-diverse hybrid by 0.032 nDCG, so the ablation hypothesis failed. V2 ships for maximum Recall@5, semantic paraphrases and evidence-type coverage: its cap prevents past proposals crowding case/code/review evidence out of writer context. This is a grounding trade-off, not a claim that fusion won. The verifier now discovers material claims even when writer metadata omits them, then removes or narrows anything unsupported before the Critic.

Judge calibration and blind-panel packets are generated but unscored. Thirty author labels and independent classmate ratings are required; the scripts intentionally emit no kappa or preference result until humans fill them. [evals/results/synthetic_rag.json; evals/results/rag_ablation.json; evals/results/judge_calibration.json]

4. R&D; C - Multi-agent orchestration

The bid graph fans Job Analyst, Risk Assessor and Fit Matcher out after load, joins their typed outputs, then routes skip/watch/bid. Bid runs continue through pricing, writer, verifier, a maximum two-pass Critic loop, compliance and approval. The conversation graph classifies client messages, proposes answers or interview slots, and interrupts before send.

Control policy	Completion	Injection recall	Mean control steps	p95 ms	Writes
Hand-built	1.00	1.00	4	7.19	0
Supervisor-style	1.00	1.00	8	7.22	0
ReAct-style	1.00	1.00	8	7.15	0

Model-backed frozen GigSim slice

Model / policy	Controller completion	Final F1	Safety violations	p95 s
Qwen / hand	0.833	1.00	0	30.0
Qwen / supervisor	1.000	1.00	0	4.1
Qwen / ReAct	0.833	1.00	0	19.0
Llama / hand	1.000	1.00	0	4.2
Llama / supervisor	1.000	1.00	0	7.5
Llama / ReAct	1.000	1.00	0	16.3

Interpretation

The public baseline policies agreed because they share deterministic nodes. The separate 36-row real-model study froze six direct/attack jobs, budgets and one retry injection. Every final action was correct and safe, but Qwen failed the controller schema once in hand and once in ReAct; the bounded fallback preserved the outcome. Llama completed all rows. ReAct used the most tokens and latency. With no decision or synthetic-outcome gain, the explicit hand-built graph remains the production boundary.

Durability and failure handling

The API seeds a Postgres START checkpoint and returns a queued thread. Celery verifies tenant/user/job identity, resumes the exact thread, and writes a ranked inbox summary. Approval state survives API restart. Every marketplace write reuses an idempotency key. Workers acknowledge late, retry bounded connection failures, and place terminal failures in a PII-redacted, bounded Redis dead-letter list. The writer tries its configured open model, then the small fallback, then a deterministic grounded fallback.

Sources: evals/results/agent_design.json; evals/results/agent_model_design.json; docs/AGENT_MODEL_DESIGN_EXPERIMENT.md; tests/integration/test_postgres_durability.py; worker/tasks.py

5. Product and production engineering

The typed Next.js application includes sign-in, persisted onboarding, a ranked inbox, job detail, live run timeline, proposal studio, pipeline, messages, analytics and settings. Users choose a persona, rates, filters and Connects limits, and can upload private PDF/Markdown/text evidence. Every claim exposes evidence on hover. Paragraph regeneration returns to verification; approvals resume the interrupted thread.

Area	Implementation evidence
Streaming	Authenticated SSE polls durable checkpoints and de-duplicates node events
API contract	FastAPI OpenAPI regenerates the TypeScript client
Responsive UI	22 Playwright cases pass: 11 flows on desktop and mobile
Background work	Celery long runs, beat saved searches, Redis dedupe/DLQ
Packaging	Compose full stack plus live Render web/API/Postgres reviewer profile
Evaluation CI	make eval plus regression thresholds; unit, contract and UI workflows

Open models and observability

The same nine fixed synthetic jobs were run per model with temperature 0, seed 4016, identical type-diverse V2 evidence and one excluded warm-up. Both candidates achieved perfect validity/grounding; the registered rule selected Qwen3 4B on personalization and latency. Llama 3.1 8B is the model fallback before the deterministic writer.

Writer	Valid/citations	Unsupported after	Personalization	p95 s
Qwen3 4B (primary)	1.00 / 1.00	0.00	0.630	11.4
Llama 3.1 8B	1.00 / 1.00	0.00	0.556	33.5

The reviewer is live at https://gigbid-web.onrender.com; its hosted graph produced five events and five Postgres checkpoints. Phoenix received a content-free OTLP span locally. Three signed-out, redacted execution snapshots are published under https://gigbid-web.onrender.com/traces for scam/injection skip, Critic/approval and crash/resume evidence. [evals/results/model_profiles.json; docs/MODEL_PROFILE_EXPERIMENT.md]

Simulator outcome study (synthetic rule engine): 90 jobs, bid/skip F1 1.00 and scam recall 1.00 for every persona; reply rates 0.667 AI/ML, 0.917 full-stack and 0.708 data; hire rates 0.583, 0.667 and 0.542. These are simulator plumbing outcomes, not real marketplace causality. [evals/results/simulator_outcomes.json]

6. Change request and signature extension

Configuration-driven policy change

Policy rules, Connects thresholds, rate limits, schedules and model routing live in versioned configuration instead of node prompts. A new platform rule is added to the policy pack and compliance evaluator, then covered by an attack test; graph topology and marketplace adapters do not change. Because the instructor's final 2 November change text was not included in the available material, this report does not fabricate a line count or elapsed time for that change.

X3 Portfolio Builder

GitHub's official MCP server is allowlisted for repository reads. The builder detects technologies in README/code, quarantines unverifiable outcome claims, and produces a pending candidate. Only explicit approval converts the code-verifiable summary into a private portfolio evidence item. The read-only Portfolio MCP is published as the versioned assignment-ii-v1.2.0 GitHub release with a documented Docker install.

Synthetic repository study (n=12)	Result
Technology precision	1.00
Technology recall	1.00
Unsupported outcome-claim recall	1.00
False-positive rate	0.00
Human approval required	100%

The fixtures test parsing and safety, not live GitHub accuracy. Live validation and reviewer acceptance need a user token and human review. [evals/results/portfolio_builder.json; docs/EXTENSION.md]

Data and provenance

The simulator uses 300 Apache-2.0 public Kaggle jobs and explicitly synthetic client preferences, gold labels and outcomes. Persona portfolios, proposals, conversations and repository snapshots are synthetic. Public contracts supply descriptive pricing priors. Private uploads are tenant-scoped and ignored by version control. Licenses, hashes, transforms and limitations are recorded in docs/DATA_CARD.md and data/sources.yaml.

7. Failure analysis

Failures were grouped by system boundary rather than hidden behind aggregate averages. This page selects twelve report-sized rows from the sixteen-class taxonomy in docs/FAILURE_ANALYSIS.md, including public-data, UI, model-reasoning and citation defects.

Failure class	Observed example	Fix / status
Missed evidence	V1 over-weighted broad repeated requirements	Keep V2; tune only on training split
Retrieval fusion	Lexical 0.833 vs type-diverse hybrid 0.801	Report loss; retain grounding/type coverage
Reasoning runaway	Default thinking ran for minutes	Disable thinking; word/token caps
Mispricing	Open-ended public budgets produced null min/max	Nullable schema and user-floor precedence
Unsupported claim	Injected 300% guarantee	Auto-discover material claims; verifier removes it
Citation contract	Models paraphrased IDs/past-proposal claims	Exact brackets, type cap, verifier rule
Compliance miss	Off-platform contact embedded in content	Policy pack plus sanitization and block reason
Prompt injection	Job tells agent to ignore rules	Untrusted delimiters, risk flag, zero write path
Tool argument	Fine-grained argument accuracy only 0.60	Strict schemas; do not trust selection alone
Marketplace failure	429/5xx/fault injection	Backoff, degraded UI, durable queue and DLQ
Model failure	Missing/unavailable local model	Small-model then deterministic grounded fallback
UI policy bug	Frontend hard-coded USD 300 floor	Inherit persisted profile; API regression test

The main lesson is that deterministic safety structure and typed contracts catch different failures than model quality. Better prompts would not have fixed nullable budgets, a hard-coded UI floor, duplicate writes or lost approval state.

8. Safety, limitations and completion boundary

What GigBid must never do

Never scrape, use session cookies, auto-submit, bypass final platform confirmation, spend Connects without approval, reveal OAuth secrets, log unmasked PII, follow job or message instructions, or state experience not supported by portfolio evidence. These constraints are enforced by port boundaries, tool allowlists, interrupts, idempotency, policy checks, claim verification, tenant namespaces and redacted audit/dead-letter logs.

Honest limitations

Pending item	Why it cannot be generated locally
30 author labels + Cohen kappa	Requires the student to inspect and label all items
Blind classmate panel	Requires independent human raters
Hidden test results	Instructor-only dataset and post-deadline run
Optional GitHub live validation	Requires the user's PAT and consented repositories
Public video and article links	The 85-second MP4 and 1,683-word draft exist locally; the student must publish them
Exact change-request diff	Final instructor policy update text was not supplied

Conclusion

GigBid's core local product is implemented and reproducible: simulator, MCP servers, two durable graphs, RAG variants, verification, worker scheduling, safety controls, typed full-stack UI, live Render reviewer and evaluation gates. The evidence supports shipping the explicit hand-built graph, type-diverse V2 retrieval and Qwen3 4B writer while preserving negative results. The remaining human studies, public video/article links and instructor hidden run must still be filled with genuine evidence.

Primary evidence index: docs/REQUIREMENTS_MATRIX.md. Reproduction: make test; make eval; make eval-tools; make eval-model-profiles; make eval-agent-design; make eval-agent-model-design; make eval-portfolio-builder; pnpm --dir apps/web test:e2e.